Tecnológico de Costa Rica

Ingeniería en Computadores

Fundamentos de Sistemas Computacionales

Energy Saving and Telemetry

Kendall Mena Arias 2025095924

Julian Arrieta Ibarra 2022116242

II Semestre Octubre del 2026

Contenido

Introducción 3

Recomendaciones 3

Conclusion 3

Reglas del Grupo Y Roles 4

- Notificación de ausencias 4
- Canal oficial único 4
- Actualización de estado 4
- Cero suposiciones 4
- Propiedad de módulos 4

Plan de Pruebas del Sistema 5

Alcance de las Pruebas 5

Criterios de Aceptación y Rechazo 5

Casos de Prueba Específicos 5

Caso de Prueba 01: Verificación de Aislamiento y Coexistencia Eléctrica 5

Caso de Prueba 02: Control de Sentido de Giro de los Motores 6

Caso de Prueba 03: Regulación de Velocidad mediante PWM 6

Caso de Prueba 04: Ejecución de Trayectoria Autónoma en Zig-Zag 6

Diagrama de Modulos 7

Literatura y Fuentes 8

Bitácora de Desarrollo del Proyecto y Fechas de las Actividades 8

Semana 1: Planteamiento y Primeras Conexiones 8

• Lunes (Día 1) — Reunión Virtual 1 8

• Miércoles (Día 3) — Trabajo Individual e Investigación 9

• Viernes (Día 5) — Reunión Presencial 1 (Laboratorio) 9

Semana 2: Integración de Motores y Ajustes Finales 9

• Miércoles (Día 10) — Trabajo Individual Irregular 9

• Viernes (Día 12) — Reunión Virtual 3 10

• Domingo (Día 14) — Reunión Presencial 2 (Ajustes Físicos y Cierre) 10

Introducción

El presente proyecto describe el diseño, desarrollo e implementación de un sistema de control electrónico personalizado para un vehículo robótico móvil de tracción diferencial. El propósito central consiste en intervenir un automóvil de juguete convencional de dos motores para remover por completo su circuito integrado de control remoto original. En su lugar, se integra una arquitectura embebida moderna basada en el microcontrolador Raspberry Pi Pico W, combinada con un puente H L298N como etapa de potencia para la gestión de los motores de corriente continua. A través de este proceso de reingeniería, se dota al vehículo de capacidades autónomas y de conectividad inalámbrica, sentando las bases para el estudio de los sistemas de cómputo físico y la interacción directa entre el software y el hardware.

Recomendaciones

- Implementar filtros de desacoplo adicionales con el fin de suprimir de manera más drástica los picos de ruido que puedan interferir con la lógica interna de la placa controladora.

- Optar por una fuente de alimentación recargable de mayor densidad energética, como baterías de iones de litio (Li-ion) configuradas en celdas 18650, para sustituir el banco de baterías AA convencionales; esto garantizará un rendimiento más lineal y un tiempo de operación extendido para los motores.

- Expandir la lógica de programación actual para aprovechar las capacidades inalámbricas embebidas en el chip Infineon CYW43439 de la Raspberry Pi Pico W, permitiendo el desarrollo de una interfaz web o el uso del protocolo MQTT para controlar el vehículo de forma remota a través de una red local.

- Integrar un sistema de sensores externos conectados a los pines GPIO disponibles, con el fin de evolucionar el prototipo de un sistema de bucle abierto preprogramado a un vehículo autónomo de bucle cerrado capaz de reaccionar a su entorno.

Conclusion

- Sustitución del circuito integrado de fábrica por una arquitectura abierta basada en la Raspberry Pi Pico W demostró ser una estrategia de reingeniería viable, permitiendo un control preciso y modular sobre la dinámica de movimiento del vehículo robótico.

- La separación física y eléctrica entre la etapa lógica (Raspberry Pi Pico W) y la etapa de potencia (controlador L298N) resultó indispensable para mitigar el ruido electromagnético y evitar caídas de voltaje en el microcontrolador causadas por la alta demanda de corriente de los motores de corriente continua.

- El uso de MicroPython en combinación con el entorno de desarrollo Visual Studio Code facilitó la programación del firmware del sistema, evidenciando que el control de periféricos de hardware mediante manipulación directa de registros GPIO es eficiente para la ejecución de trayectorias preprogramadas.

- El prototipo final cumplió de manera exitosa con los requerimientos de tracción diferencial, validando que es posible transformar hardware de consumo genérico en una plataforma de experimentación tecnológica robusta para el estudio de sistemas de cómputo físico.

Reglas del Grupo Y Roles

- **Notificación de ausencias**: Si un integrante no puede asistir a una sesión de trabajo, debe avisar con al menos 4 horas de anticipación y justificar su falta.

- **Canal oficial único**: Toda la coordinación técnica, acuerdos y cambios en el diseño se centralizarán en un solo canal de comunicación

- **Actualización de estado**: Al inicio de cada semana, cada integrante debe reportar brevemente el avance de su asignación (diseño, código, conexiones o documentación) y alertar inmediatamente si está atascado con algún problema.

- **Cero suposiciones**: Si hay dudas sobre cómo conectar un componente o cómo estructurar una función del código, se debe consultar al equipo o revisar la hoja de datos (datasheet) antes de proceder.

- **Propiedad de módulos**: El proyecto se dividirá en tres áreas principales (Hardware, Software y Documentación escrita).

Plan de Pruebas del Sistema

Alcance de las Pruebas

El alcance de este plan de pruebas abarca la verificación y validación del comportamiento integrado del vehículo robótico móvil. Se contemplan las pruebas unitarias de los componentes electrónicos, las pruebas de integración entre la etapa lógica (Raspberry Pi Pico W) y la etapa de potencia (Puente H L298N), y finalmente las pruebas de sistema que evalúan la autonomía y el desplazamiento físico del prototipo en un entorno controlado.

Criterios de Aceptación y Rechazo

- Criterio de Aceptación: El sistema se considerará aprobado si el microcontrolador ejecuta el firmware sin reinicios por caída de voltaje, los motores responden fielmente a los cambios de sentido y velocidad modulados por PWM, y el vehículo completa las trayectorias autónomas preprogramadas dentro de un margen de desviación lineal menor al 10%.
- Criterio de Rechazo: El sistema será rechazado si se presentan retornos de corriente hacia la Raspberry Pi Pico W, si existe una desincronización crítica en los motores que impida el avance en línea recta, o si el acoplamiento mecánico interrumpe el desplazamiento continuo del vehículo.

Casos de Prueba Específicos

Caso de Prueba 01: Verificación de Aislamiento y Coexistencia Eléctrica

- Objetivo: Garantizar la estabilidad del sistema al energizar ambas etapas simultáneamente.
- Pasos a ejecutar:
    1. Conectar el banco de baterías AA a las borneras de alimentación del puente H L298N.
    2. Conectar la Raspberry Pi Pico W a la computadora mediante el cable USB.
    3. Medir con el multímetro el voltaje en los pines de tierra (GND) comunes para verificar estabilidad.
- Resultado esperado: Ambas fuentes de alimentación coexisten de manera segura. No se observan caídas de voltaje en la lógica ni calentamiento anómalo en los reguladores.

Caso de Prueba 02: Control de Sentido de Giro de los Motores

- Objetivo: Validar que las señales lógicas digitales controlen correctamente la dirección de los motores.
- Pasos a ejecutar:
    1. Cargar el script de prueba en MicroPython para alternar los pines GPIO de dirección (IN1, IN2, IN3, IN4).
    2. Ejecutar comandos secuenciales de: Marcha adelante (2s), Parada (1s), Marcha atrás (2s).
- Resultado esperado: Los motores de corriente continua invierten su polaridad magnética al unísono, respondiendo de forma inmediata a los cambios lógicos del programa.
-

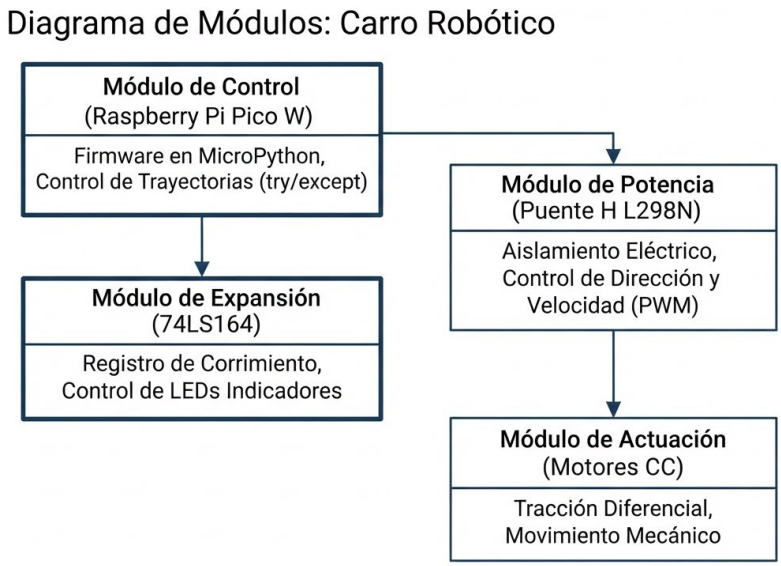
Caso de Prueba 03: Regulación de Velocidad mediante PWM

- Objetivo: Evaluar la respuesta de los motores ante la Modulación por Ancho de Pulsos.
- Pasos a ejecutar:
    1. Inicializar los pines habilitadores (ENA y ENB) con una frecuencia base de 1000 Hz.
    2. Incrementar de forma progresiva el ciclo de trabajo (Duty Cycle) desde 30% hasta el 100% de potencia.
- Resultado esperado: Los motores modifican su par de fuerza y revoluciones por minuto de manera lineal, evidenciando un cambio de velocidad perceptible en el prototipo.

Caso de Prueba 04: Ejecución de Trayectoria Autónoma en Zig-Zag

- Objetivo: Confirmar la precisión del algoritmo de control diferencial en un entorno físico.
- Pasos a ejecutar:
    1. Programar un retraso (delay) inicial de 30 segundos tras el encendido para permitir la colocación del carro en la pista.
    2. Iniciar la rutina secuencial de avance recto y giros alternados simulando una ruta en zig-zag.
- Resultado esperado: El vehículo respeta el tiempo de espera, avanza de forma estable y ejecuta los giros pivotando sobre su propio eje según las temporizaciones estipuladas en el código.

Diagrama de Modulos



ARQUITECTURA MODULAR DEL SISTEMA

El diseño del vehículo robótico se rige bajo un enfoque de arquitectura modular, lo que permite aislar las funciones de software de los componentes de hardware para facilitar el mantenimiento y la escalabilidad del sistema. El diagrama de módulos se compone de las siguientes secciones interconectadas:

- Módulo de Control (Firmware): Desarrollado en MicroPython e implementado en la Raspberry Pi Pico W. Este bloque contiene el algoritmo principal encargado de la temporización y la toma de decisiones para las trayectorias autónomas, incorporando rutinas de seguridad mediante control de excepciones (try/except).

- Módulo de Potencia: Representado por el circuito integrado L298N. Su única función es actuar como una interfaz de aislamiento eléctrico, recibiendo señales digitales de baja potencia del módulo de control y traduciéndolas en salidas de alta corriente aptas para activar los motores CC.

- Módulo de Expansión de Salidas: Compuesto por el registro de corrimiento 74LS164. Este módulo procesa datos seriales provenientes del microcontrolador y los convierte en salidas paralelas para manipular una barra de LEDs indicadores, reduciendo el consumo de pines GPIO físicos en la placa central.

- Módulo de Actuación Mecánica: Constituido por los dos motores de corriente continua en configuración diferencial, encargados de transformar la energía eléctrica regulada por el puente H en movimiento cinético.

Literatura y Fuentes

STMicroelectronics. (2000). L298 dual full-bridge driver datasheet. https://www.st.com/resource/en/datasheet/l298.pdf

Texas Instruments. (1988). SN74LS164 8-bit parallel-out serial shift registers datasheet. https://www.ti.com/lit/ds/symlink/sn74ls164.pdf

Raspberry Pi Ltd. (2022). Raspberry Pi Pico W datasheet: An ARM Cortex-M0+ based microcontroller with wireless connectivity. Raspberry Pi Documentation. https://datasheets.raspberrypi.com/picow/pico-w-datasheet.pdf

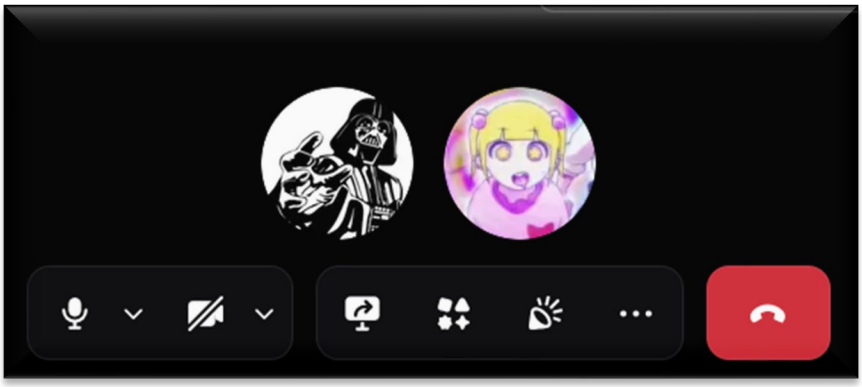
Bitácora de Desarrollo del Proyecto y Fechas de las Actividades

Semana 1: Planteamiento y Primeras Conexiones

• Lunes (Día 1) — Reunión Virtual 1

• Actividad: Sesión inicial de coordinación para definir el alcance del proyecto, revisar los requerimientos de la guía y distribuir los componentes básicos (Raspberry Pi Pico W, registro de corrimiento 74LS164 y driver L298N).

• Acuerdos: Se estableció un cronograma preliminar (sujeto a la disponibilidad por exámenes de otros cursos) y se dividió el trabajo en dos fases: control de luces/sensores y control de potencia de motores.



• ***Miércoles (Día 3) — Trabajo Individual e Investigación***

• Actividad: Lectura de las hojas de datos (datasheets) del integrado 74LS164 para comprender la distribución de pines y la lógica de desplazamiento de bits. Configuración inicial del entorno de desarrollo (Thonny IDE) en las computadoras personales.

• ***Viernes (Día 5) — Reunión Presencial 1 (Laboratorio)***

• Actividad: Ensamblaje del primer circuito impreso/protoboard. Conexión de la Raspberry Pi Pico W junto con la fotorresistencia y el registro 74LS164 para el control de los LEDs (simulación de luces delanteras y traseras en paralelo).

• Problema detectado: Al cargar el script inicial en MicroPython, los LEDs parpadeaban de forma errática o se encendían de manera aleatoria al recibir datos del integrado.

• Solución: Se revisó el cableado con el osciloscopio del laboratorio y se detectó ruido electromagnético por falta de desacoplo y un pin de reloj (CLK) mal asignado en el código. Se reasignaron los pines GPIO y se añadieron resistencias de sintonía adecuadas (330 ohms), estabilizando la señal de datos secuencial.

Semana 2: Integración de Motores y Ajustes Finales

• ***Lunes (Día 8) — Reunión Prescencial 2***

• Actividad: Revisión del código de control de luces tras la solución del laboratorio. Programación modular de las funciones encargadas de leer el comportamiento de la fotorresistencia para encender o apagar todo el bloque de LEDs de forma automática.



• ***Miércoles (Día 10) — Trabajo Individual Irregular***

• Actividad: Montaje independiente del driver de motores L298N para el control de tracción del prototipo. Redacción de las funciones PWM básicas en MicroPython para variar la velocidad.

• Problema detectado: Al probar el conjunto completo, los motores consumían demasiada energía, provocando que la Raspberry Pi Pico W se reiniciara constantemente y perdiendo la conexión serial.

• Solución: Se aisló la etapa de potencia utilizando una fuente de alimentación externa (baterías independientes) para el driver L298N, unificando únicamente las tierras (GND) del circuito para mantener la referencia. Con esto se evitó el reinicio del microcontrolador.

• Viernes (Día 12) — Reunión Virtual 2

• Actividad: Integración de los módulos de software en un único script principal (main.py). Optimización de los tiempos de espera para que la lectura de la fotorresistencia no bloqueara las instrucciones de movimiento de los motores.

• Domingo (Día 14) — Reunión Presencial 3 (Ajustes Físicos y Cierre)

• Actividad: Montaje final sobre el chasis del vehículo. Pruebas de rodamiento en pista y validación del encendido automático de luces en zonas oscuras.

• Resultado: El prototipo cumple con la lógica solicitada, respondiendo de manera estable tanto en los cambios de dirección como en la activación secuencial de sus componentes periféricos. Bitácora cerrada para entrega del proyecto.